Numerical Addendum: HydrationShell Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

This calculator does not run a factorization, a quadrature rule, a tensor projection, or a spatial query over waters. The implementation uses direct loops over explicit waters and ions: one squared-distance shell gate, one sign test against a reference direction, one water-dipole direction-cosine average, and a nearest-ion minimum. The sibling `HydrationGeometryResult` uses the same first-shell water scan but takes its reference direction from the solvent-accessible surface area (SASA) normal, sums first-shell water dipoles, and derives alignment and coherence from that sum. The listings below are distilled from the source — loops and bookkeeping trimmed — but the constants, guards, inequalities, and output shapes are the implementation's.

1 Candidate enumeration

The applied-maths inventory lists nanoflann k-d trees as the shared spatial query mechanism. This pair of classes does not use a water k-d tree. The `HydrationShellResult` header declares `SpatialIndexResult` as a pipeline dependency, but `Compute()` never reads it; `SpatialIndexResult` itself builds trees over protein atom positions, ring centres, and bond midpoints, not over `SolventEnvironment::water_O_positions`. The water part of the calculator is therefore an all-waters scan for every protein atom. The ion part is an all-ions scan for every protein atom. For N protein atoms, W waters, and I ions, the work in these two loops is $O(NW + NI)$ per frame.

The first-shell gate is a squared-distance comparison. For atom i at $\boldsymbol{r}_i$ and water oxygen w at $\boldsymbol{o}_w$, `r` is $\boldsymbol{o}_w - \boldsymbol{r}_i$ and `d_sq` is $|\boldsymbol{r}|^2$. The configured cutoff is `water_first_shell_cutoff`, $3.5\,$Å in `data/calculator_params.toml`. Since the code continues only when `d_sq > first_sq`, a water exactly on the cutoff remains in the shell. There is no radial histogram and no second shell in these classes.

```
const double first_shell_cutoff =
    CalculatorConfig::Get("water_first_shell_cutoff");
const double first_sq = first_shell_cutoff * first_shell_cutoff;

for (size_t ai = 0; ai < N; ++ai) {
    Vec3 pos_i = conf.AtomAt(ai).Position();

    for (size_t wi = 0; wi < W; ++wi) {
        Vec3 r = solvent.waters[wi].O_pos - pos_i;  // atom -> oxygen
        double d_sq = r.squaredNorm();
        if (d_sq > first_sq) continue;              // accepts d <= cutoff

        double d = std::sqrt(d_sq);
        Vec3 r_hat = r / d;
        ...
    }
}
```

The shell gate depends only on the oxygen position. Hydrogen positions enter only through water dipoles after the oxygen has passed the gate. The normalisation `r / d` has no zero-distance guard; a selected water oxygen exactly coincident with the target atom is outside the guarded cases in this path.

2 Centroid half-shell and water dipoles

`HydrationShellResult` first forms `protein_com`, but the number is a coordinate centroid, not a mass centre:

$$\boldsymbol{c} = \frac{1}{N}\sum_{j=1}^{N} \boldsymbol{r}_j.$$

For each atom, `to_interior` is $\hat{\boldsymbol{u}}_i = (\boldsymbol{c} - \boldsymbol{r}_i)/|\boldsymbol{c} - \boldsymbol{r}_i|$, the centroid-facing reference direction. A first-shell water is buried when $\hat{\boldsymbol{r}}_{iw} \cdot \hat{\boldsymbol{u}}_i > 0$. The equality case goes to exposed, because every non-positive value follows the `else` branch. The normalisation of $\boldsymbol{c} - \boldsymbol{r}_i$ is also unguarded; an atom exactly at the coordinate centroid is outside the guarded cases in this path.

```
Vec3 protein_com = Vec3::Zero();
for (size_t i = 0; i < N; ++i)
    protein_com += conf.AtomAt(i).Position();
protein_com /= static_cast<double>(N);

Vec3 to_interior = (protein_com - pos_i).normalized();

int n_exposed = 0, n_buried = 0;
double dipole_cos_sum = 0.0;
int dipole_count = 0;

double cos_interior = r_hat.dot(to_interior);
if (cos_interior > 0)
    ++n_buried;
else
    ++n_exposed;

Vec3 dipole = solvent.waters[wi].Dipole();
double dip_mag = dipole.norm();
if (dip_mag > CalculatorConfig::Get("near_zero_vector_norm_threshold")) {
    dipole_cos_sum += dipole.dot(r_hat) / dip_mag;
    ++dipole_count;
}
```

The half-shell output is an exposed-side fraction, not a signed centred asymmetry:

$$h_i = \begin{cases} n_{\text{exposed}}/(n_{\text{exposed}} + n_{\text{buried}}), & n_{\text{exposed}} + n_{\text{buried}} > 0, \\ 0, & n_{\text{exposed}} + n_{\text{buried}} = 0. \end{cases}$$

The first-shell count itself is not written by `HydrationShellResult`, so $h_i = 0$ can mean either no first-shell water or all counted waters on the buried side.

`WaterMolecule::Dipole()` returns

$$\boldsymbol{\mu}_w = q_{\mathrm{H}}(\boldsymbol{h}_{1w} - \boldsymbol{o}_w) + q_{\mathrm{H}}(\boldsymbol{h}_{2w} - \boldsymbol{o}_w),$$

because the oxygen is the origin of the water-local displacement sum. The coordinates are in Å and the charges in e. Thus $\boldsymbol{\mu}_w$ is in e Å.

For the HydrationShell dipole channel, each accepted water contributes only the direction cosine $\boldsymbol{\mu}_w \cdot \hat{\boldsymbol{r}}_{iw}/|\boldsymbol{\mu}_w|$. Waters whose dipole magnitude is not greater than `near_zero_vector_norm_threshold`, 10^{-10}, still count in the half-shell fraction but do not enter this average. If no water contributes to the dipole average, the output is zero.

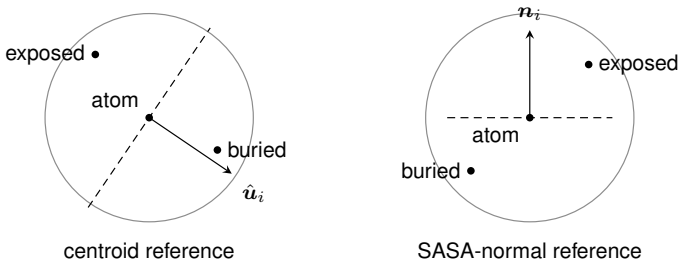

Figure. Each panel shows a first-shell ball cut by a plane through the atom. The centroid-reference panel counts the positive side of $\hat{u}_i$ as buried; the SASA-normal panel counts the positive side of n_i as exposed.

3 Nearest ion

The nearest-ion channel is another direct scan. For each ion k, the code computes $|z_k - r_i|$ in Å. The stored best value changes only when the new distance is strictly smaller than the current best, so an exact tie keeps the earlier ion in storage order. After the scan, the cutoff test is also strict: an ion at exactly `hydration_ion_cutoff`, 20.0 Å, is reported as absent.

```
double best_ion_dist = std::numeric_limits<double>::max();
double best_ion_charge = 0.0;

for (size_t ii = 0; ii < solvent.IonCount(); ++ii) {
    double d = (solvent.ions[ii].pos - pos_i).norm();
    if (d < best_ion_dist) {
        best_ion_dist = d;
        best_ion_charge = solvent.ions[ii].charge;
    }
}

atom.nearest_ion_distance =
    (best_ion_dist < ion_cutoff)
    ? best_ion_dist : std::numeric_limits<double>::infinity();
atom.nearest_ion_charge =
    (best_ion_dist < ion_cutoff) ? best_ion_charge : 0.0;
```

With solvent present but no ions, `best_ion_dist` stays at the largest finite double, fails the strict cutoff, and the same absent record is written: $+\infty$ for distance and zero for charge. The zero charge is interpretable only with the distance channel; it is also the absent sentinel.

4 SASA-normal hydration geometry

`HydrationGeometryResult` keeps the same first-shell oxygen gate and the same all-waters scan, but it consumes a different reference vector: `atom.sasa_normal`, already written by `SasaResult`. This class does not recompute the sampled SASA normal; it copies the stored vector to `water_surface_normal` and uses its sign in the half-shell split. When the normal norm is not greater than `near_zero_vector_norm_threshold`, all first-shell waters count as exposed. When the normal is present, $\hat{r}_{iw} \cdot n_i > 0$ is exposed and the equality case is buried.

```
Vec3 normal = atom.sasa_normal;
atom.water_surface_normal = normal;

Vec3 dipole_sum = Vec3::Zero();
int n_exposed = 0, n_buried = 0, n_shell = 0;
```

```
for (size_t wi = 0; wi < W; ++wi) {
    Vec3 r = solvent.waters[wi].O_pos - pos_i;
    double d_sq = r.squaredNorm();
    if (d_sq > first_sq) continue;

    ++n_shell;
    dipole_sum += solvent.waters[wi].Dipole();

    double d = std::sqrt(d_sq);
    Vec3 r_hat = r / d;
    if (normal.norm() > near_zero) {
        double cos_normal = r_hat.dot(normal);
        if (cos_normal > 0) ++n_exposed;
        else                ++n_buried;
    } else {
        ++n_exposed;
    }
}
```

The net dipole $\boldsymbol{D}_i = \sum_{w \in \mathcal{W}_i} \boldsymbol{\mu}_w$ keeps the $\mathrm{e}\,$Å units of the water dipoles. No water is removed from this vector sum by a near-zero dipole guard. Three scalar channels are then derived from the same counts and vector:

$$h_i^{\mathrm{SASA}} = \frac{n_{\mathrm{exposed}}}{n_{\mathrm{exposed}} + n_{\mathrm{buried}}} \quad \text{when the denominator is nonzero,}$$

$$a_i = \frac{\boldsymbol{D}_i \cdot \boldsymbol{n}_i}{|\boldsymbol{D}_i|\,|\boldsymbol{n}_i|}, \qquad c_i = \frac{|\boldsymbol{D}_i|}{n_{\mathrm{shell}}}.$$

The code writes zero for h_i^{SASA} when no water is in the shell, zero for a_i when either $|\boldsymbol{D}_i|$ or $|\boldsymbol{n}_i|$ is not greater than the near-zero threshold, and zero for c_i when the shell is empty. `sasa_dipole_alignment` is a cosine. `sasa_dipole_coherence` is not dimensionless; it has units of $\mathrm{e}\,$Å.

```
atom.sasa_first_shell_count = n_shell;
atom.sasa_half_shell_asymmetry =
    (n_exposed + n_buried > 0)
    ? static_cast<double>(n_exposed) / (n_exposed + n_buried)
    : 0.0;

atom.water_dipole_vector = dipole_sum;

double dip_mag = dipole_sum.norm();
double norm_mag = normal.norm();
if (dip_mag > near_zero && norm_mag > near_zero)
    atom.sasa_dipole_alignment =
        dipole_sum.dot(normal) / (dip_mag * norm_mag);
else
    atom.sasa_dipole_alignment = 0.0;

atom.sasa_dipole_coherence =
    (n_shell > 0) ? dip_mag / static_cast<double>(n_shell) : 0.0;
```

5 Output shapes

No tensor is emitted by either class, so the shared `SphericalTensor::Decompose` path is not used here. The values are scalars, counts, and two three-vectors. They are solvent-geometry descriptors, not shielding tensors and not chemical shifts.

```
// hydration_shell.npy: (N, 4)
data[i*4 + 0] = a.half_shell_asymmetry;
data[i*4 + 1] = a.mean_water_dipole_cos;
data[i*4 + 2] = a.nearest_ion_distance;
```

```
data[i*4 + 3] = a.nearest_ion_charge;

// water_polarization.npy: (N, 10)
data[i*10 + 0] = a.water_dipole_vector.x();
data[i*10 + 1] = a.water_dipole_vector.y();
data[i*10 + 2] = a.water_dipole_vector.z();
data[i*10 + 3] = a.water_surface_normal.x();
data[i*10 + 4] = a.water_surface_normal.y();
data[i*10 + 5] = a.water_surface_normal.z();
data[i*10 + 6] = a.sasa_half_shell_asymmetry;
data[i*10 + 7] = a.sasa_dipole_alignment;
data[i*10 + 8] = a.sasa_dipole_coherence;
data[i*10 + 9] = static_cast<double>(a.sasa_first_shell_count);
```

The count in column 9 of `water_polarization.npy` is stored as a double because the NPY writer call emits a float64 matrix. The nearest-ion distance in `hydration_shell.npy` may be $+\infty$. The code writes no explicit boolean ion-present channel in this file.

6 Glossary

First hydration shell. A ball around a protein atom, populated by water oxygens rather than by all water charge sites. A water with oxygen exactly $3.5\,\text{Å}$ from the atom is included by the squared-distance test; its hydrogens then contribute only through the dipole vector. Formally, it is $\{w : |\boldsymbol{o}_w - \boldsymbol{r}_i|^2 \le R_w^2\}$ with $R_w =$ `water_first_shell_cutoff`.

Half-shell asymmetry. A fraction made by cutting the first-shell ball into two half-spaces through the atom and counting waters on one named side. In `HydrationShellResult` the exposed side is the non-positive side of the centroid-facing direction; in `HydrationGeometryResult` it is the positive side of the SASA normal. If that normal's norm is not greater than the near-zero threshold, all waters count as exposed. Formally, it is $n_{\text{exposed}}/(n_{\text{exposed}} + n_{\text{buried}})$, with zero written for an empty denominator.

Water dipole vector. The arrow from the water oxygen toward the charged hydrogen side, scaled by the topology hydrogen charge and the O–H geometry. For a three-site water with topology hydrogen charge q_{H}, the vector is the sum of the two charged O–H displacements, in $\text{e}\,\text{Å}$. Formally, $\boldsymbol{\mu}_w = \sum_j q_j(\boldsymbol{r}_j - \boldsymbol{o}_w)$, with the oxygen term zero in the stored origin.

SASA normal. The outward direction already estimated by the SASA calculator: it averages the unit directions of the non-occluded test points on an expanded atom sphere and normalises the sum when that sum is above the near-zero threshold. If the stored normal's norm is not above that threshold, HydrationGeometry then takes the no-direction branch. Formally, this calculator consumes $\boldsymbol{n}_i =$ `atom.sasa_normal` as stored; it does not rebuild the Fibonacci-lattice sampling and occlusion test that produced it.